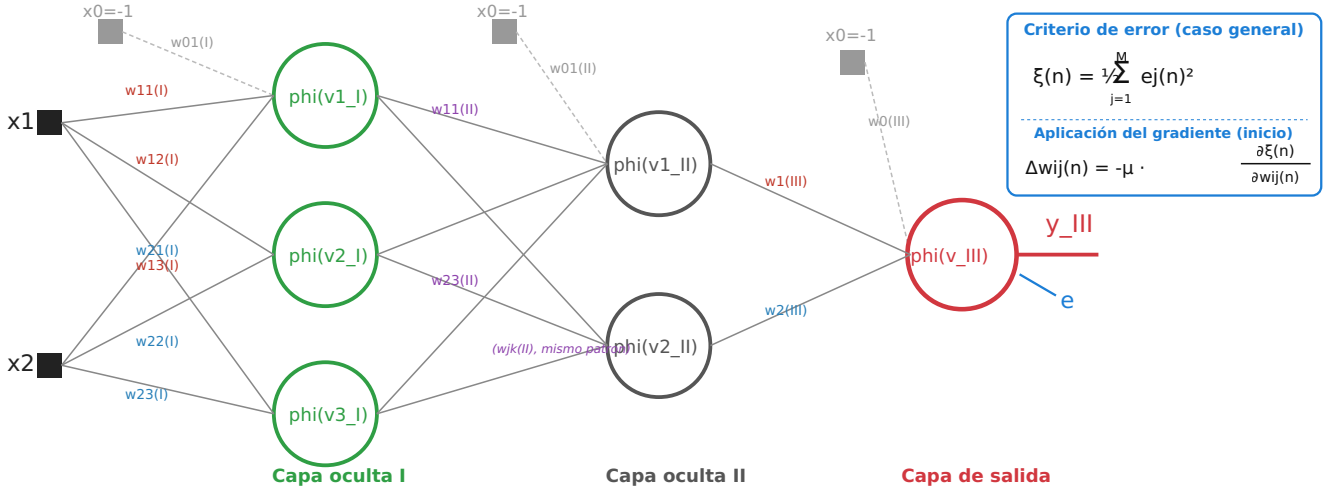
BACK PROPAGATION

Retropropagacion en el Perceptron Multicapa — arquitectura, algoritmo y formulas generales

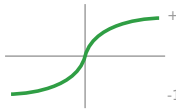


$w_{ij}^{(p)}$ = peso que conecta la neurona **i** de la capa anterior (origen) con la neurona **j** de la capa **p** (destino) | $w_{0j}^{(p)}$ = peso del bias (x0=-1) de esa capa

Algoritmo general

1. **Inicializacion** — pesos al azar
2. **Propagacion hacia adelante** — calcular v e y, capa por capa
3. **Propagacion hacia atras** — calcular los delta desde la salida hacia la entrada
4. **Adaptacion de los pesos** — aplicar la regla delta en cada capa
5. **Iteracion** — volver al paso 2 hasta converger

¿Qué función se deriva? — La sigmoide (simétrica)



$y = \varphi(v) = 2/(1+e^{-v}) - 1$

$\varphi'(v) = ½(1+y)(1-y)$

Formulas generales

1. Propagacion hacia adelante

$$v_j^{(p)} = \Sigma\, w_{ij}^{(p)} \cdot x_i \qquad y_j^{(p)} = \varphi(v_j^{(p)})$$

(en capas ocultas, $x = y_i^{(p-1)}$, salida de la capa anterior)

2. Error — capa de salida (L)

$$e_j = d_j - y_j^{(L)}$$

3. Delta — capa de salida (L)

$$\delta_j^{(L)} = e_j \cdot ½(1+y_j^{(L)})(1-y_j^{(L)})$$

4. Delta — capa oculta (p)

$$\delta_j^{(p)} = (\Sigma_k\, \delta_k^{(p+1)} \cdot w_{jk}^{(p+1)}) \cdot ½(1+y_j^{(p)})(1-y_j^{(p)})$$

5. Actualizacion de pesos

$$w_{ij}^{(p)}(n+1) = w_{ij}^{(p)}(n) + \eta \cdot \delta_j^{(p)} \cdot x_i$$

(en capas ocultas, $x = y_i^{(p-1)}$, salida de la capa anterior)

Regla delta generica (la formula ancla): **$\Delta w = \eta \cdot \delta \cdot y$**

x0=-1 en cada capa (bias) | p=capa actual | p-1=capa anterior | p+1=capa siguiente | L=última capa (salida) | n=iteración de entrenamiento | η=velocidad de aprendizaje
i=neurona de la capa anterior (p-1) | j=neurona de la capa actual (p) | k=neurona de la capa siguiente (p+1)